

Script Berbahaya di Balik Kolom Komentar: Eksperimen XSS (Cross-Site Scripting) dan Cara Mencegahnya

Alipiani Dwi Putri

NIM 312410691 | Kelas I241B

Program Studi Teknik Informatika, Universitas Pelita Bangsa

PENDAHULUAN

Di era digitalisasi yang semakin pesat, aplikasi web telah menjadi tulang punggung berbagai layanan modern mulai dari perbankan daring, media sosial, platform e-commerce, hingga sistem informasi akademik. Namun di balik kemudahan yang ditawarkan, terdapat ancaman serius yang seringkali luput dari perhatian para pengembang: kerentanan keamanan pada sisi klien (client-side vulnerability).

Cross-Site Scripting atau yang dikenal dengan singkatan XSS adalah salah satu ancaman keamanan web yang paling berbahaya dan paling banyak ditemukan hingga saat ini. OWASP (Open Web Application Security Project) dalam laporan OWASP Top 10 2021 mengkategorikan XSS sebagai bagian dari kelompok Injection yang menduduki posisi ketiga sebagai kerentanan web paling kritis secara global (OWASP Foundation, 2021). Fakta ini menunjukkan bahwa meskipun teknik serangan ini sudah dikenal luas, praktik pengembangan yang tidak aman masih terus terjadi.

XSS bekerja dengan cara menyisipkan kode JavaScript berbahaya ke dalam halaman web yang kemudian dieksekusi oleh browser korban. Berbeda dengan SQL Injection yang menyerang basis data di sisi server, XSS menyerang langsung di sisi pengguna mencuri sesi login, mengambil data cookie, memanipulasi tampilan halaman, bahkan mengarahkan pengguna ke situs berbahaya, semua tanpa sepengetahuan korban.

Artikel ini disusun berdasarkan eksperimen langsung yang dilakukan penulis menggunakan lingkungan pengembangan lokal berbasis XAMPP dan Visual Studio Code. Aplikasi web sederhana dengan kolom komentar sengaja dibangun tanpa proteksi untuk mendemonstrasikan bagaimana serangan XSS bekerja secara nyata, kemudian dilanjutkan dengan implementasi mitigasi yang terbukti efektif. Tujuan penulisan artikel ini adalah: (1) menjelaskan konsep dan mekanisme kerja XSS secara komprehensif, (2)

mendemonstrasikan tiga jenis serangan XSS melalui eksperimen nyata, (3) menganalisis akar permasalahan dari perspektif kode sumber, serta (4) mengimplementasikan dan membuktikan efektivitas mitigasi.

PEMBAHASAN UTAMA

A. Konsep Dasar Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) adalah jenis serangan injeksi di mana penyerang menyisipkan skrip berbahaya, umumnya berupa kode JavaScript, ke dalam halaman web yang kemudian dieksekusi oleh browser pengguna lain yang mengunjungi halaman tersebut. Serangan ini terjadi ketika aplikasi web menerima input dari pengguna dan menampilkannya kembali ke halaman web tanpa proses validasi atau sanitasi yang memadai (Gustiyono et al., 2024).

Untuk memahami cara kerja XSS, bayangkan sebuah kolom komentar sederhana. Ketika pengguna mengirimkan komentar berupa teks biasa seperti "Artikel ini sangat bermanfaat!", server menyimpan dan menampilkannya sebagai teks. Namun ketika penyerang mengirimkan komentar berupa kode JavaScript seperti `<script>alert('XSS')</script>`, jika tidak ada sanitasi, server akan menyimpan dan menampilkan kode tersebut sebagai bagian dari HTML halaman. Browser pengguna lain yang membuka halaman itu akan mengeksekusi kode JavaScript tersebut seolah-olah ia adalah bagian sah dari halaman web.

Berdasarkan mekanisme eksploitasinya, XSS diklasifikasikan ke dalam tiga kategori utama:

Jenis XSS	Mekanisme	Tingkat Bahaya
Reflected XSS	Payload dikirim via URL/form, langsung direfleksikan server ke halaman respons tanpa disimpan	Tinggi
Stored XSS	Payload disimpan di database/server dan dieksekusi setiap kali halaman dikunjungi oleh siapapun	Sangat Tinggi
DOM-based XSS	Payload dieksekusi melalui manipulasi Document Object Model (DOM) di sisi klien tanpa melibatkan server	Tinggi

Eksperimen dalam artikel ini berfokus pada Reflected XSS karena jenis ini paling mudah divisualisasikan, paling sering ditemukan di aplikasi web nyata, dan paling representatif untuk memahami dampak langsung dari kerentanan input tanpa sanitasi.

B. Eksperimen yang Dilakukan

Seluruh pengujian dilakukan secara eksklusif dalam lingkungan pengembangan lokal yang terisolasi. Tidak ada target, sistem, atau server pihak lain yang terlibat dalam eksperimen ini.

Persiapan Lingkungan Eksperimen

Komponen	Spesifikasi
Sistem Operasi	Windows 11
Web Server	XAMPP (Apache 2.4.58 + PHP 8.2.12)
Editor Kode	Visual Studio Code
Bahasa Pemrograman	PHP & JavaScript
Browser	Google Chrome
Aplikasi Target	Aplikasi web buatan sendiri (kolom komentar PHP)

Penulis membangun dua halaman PHP dari nol menggunakan VS Code: `index.php` sebagai versi yang sengaja dibuat rentan (tanpa sanitasi input), dan `aman.php` sebagai versi yang telah diproteksi menggunakan fungsi `htmlspecialchars()`. Kedua halaman dijalankan melalui XAMPP di alamat `localhost/xss-eksperimen/`.

⚠ VERSI RENTAN - Tanpa Sanitasi

Kolom Komentar

Tulis Komentar

Nama:

Pesan:

Tulis pesan...

Kirim Komentar

Gambar 1. Tampilan `index.php` — halaman kolom komentar versi rentan berjalan di

Kode Sumber yang Rentan (index.php)

Berikut adalah inti kode PHP yang sengaja dibuat rentan variabel input langsung diinterpolasi ke dalam HTML tanpa proses sanitasi apapun:

```
// KODE PHP RENTAN - TANPA SANITASI
$nama = $_POST['nama']; // Input langsung diambil tanpa filter
$pesan = $_POST['pesan']; // Input langsung diambil tanpa filter

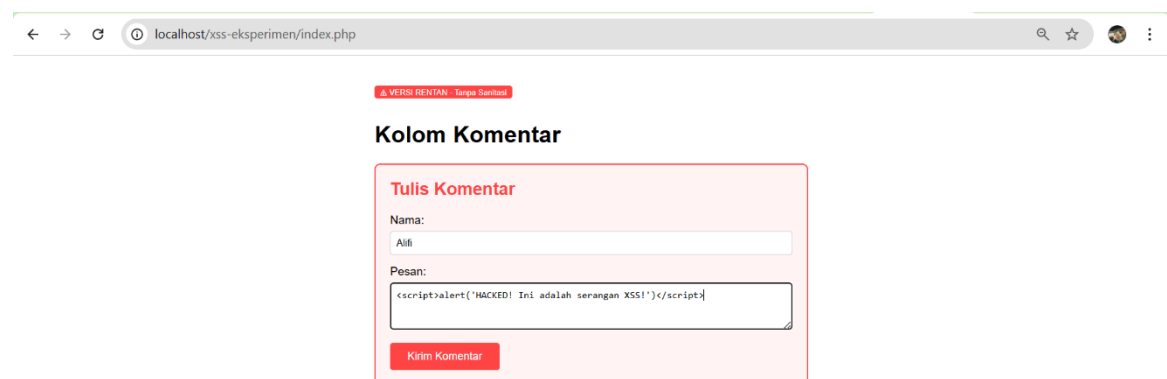
// Ditampilkan langsung ke HTML - BERBAHAYA!
echo '<strong>' . $nama . '</strong>';
echo '<p>' . $pesan . '</p>';
```

Eksekusi Serangan XSS —Tiga Tahap

Tahap 1 Serangan Dasar (Alert Popup)

Pada tahap pertama, penulis memasukkan payload berikut pada kolom Pesan:

```
<script>alert('HACKED! Ini adalah serangan XSS!')</script>
```



Gambar 2. Form diisi payload XSS pada kolom Pesan sebelum dikirim



Gambar 3. Pop-up alert muncul — bukti JavaScript berhasil dieksekusi browser

Tahap 2 Pencurian Cookie Sesi

Pada tahap kedua, payload yang lebih berbahaya diujikan untuk mendemonstrasikan kemampuan XSS mencuri informasi sensitif:

```
<script>alert('Cookie kamu: ' + document.cookie)</script>
```



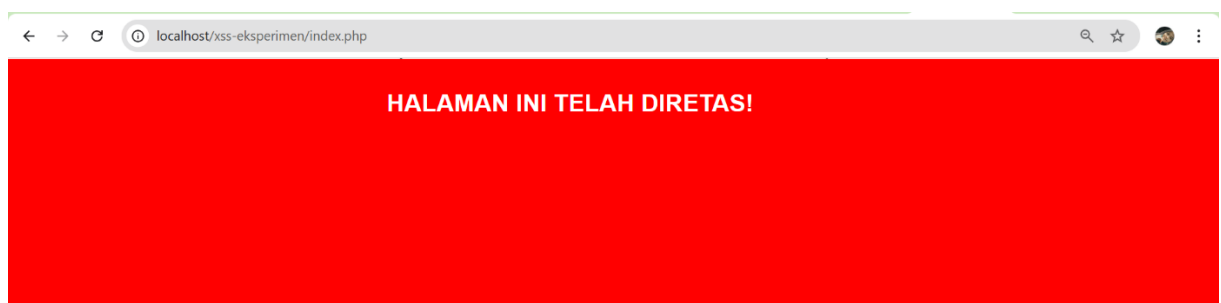
Gambar 4. Alert menampilkan nilai cookie sesi yang berhasil dicuri

Pada aplikasi web nyata yang menggunakan session management, cookie yang berhasil dicuri dapat digunakan penyerang untuk melakukan session hijacking mengambil alih akun korban tanpa perlu mengetahui username dan password.

Tahap 3 Defacement Halaman (Manipulasi Tampilan)

Pada tahap ketiga, payload yang memanipulasi seluruh tampilan halaman web dieksekusi:

```
<script>
  document.body.style.backgroundColor='red';
  document.body.innerHTML='<h1 style=color:white>HALAMAN INI TELAH
DIRETAS!</h1>';
</script>
```



Gambar 5. Halaman berubah total menjadi merah — demonstrasi defacement via XSS

C. Hasil Eksperimen dan Analisis

Ketiga tahap serangan berhasil dilaksanakan dengan sempurna. Hasil eksperimen secara ringkas dapat dilihat pada tabel berikut:

Tahap	Payload	Hasil	Dampak Nyata
1	alert('HACKED!')	Pop-up berhasil muncul	Konfirmasi kerentanan XSS
2	alert(document.cookie)	Cookie ditampilkan	Pencurian sesi pengguna
3	Manipulasi DOM	Halaman berubah total	Defacement / penipuan visual

Analisis Kode Sumber yang Rentan

Akar permasalahan dari seluruh eksperimen di atas hanya satu: variabel input dari pengguna dimasukkan langsung ke dalam output HTML tanpa proses sanitasi. Ini adalah kesalahan fundamental yang disebut sebagai "data-code confusion" ketika data dari sumber eksternal diperlakukan setara dengan instruksi program (Gustiyono et al., 2024).

Ketika penyerang memasukkan tag `<script>`, browser tidak dapat membedakan apakah skrip tersebut berasal dari pengembang atau dari input pengguna yang berbahaya. Seluruhnya diperlakukan sama dieksekusi sebagai kode JavaScript yang sah.

Dampak potensial serangan XSS pada aplikasi nyata jauh lebih serius, mencakup: pencurian kredensial login melalui session hijacking, penyebaran malware kepada seluruh pengunjung halaman, manipulasi konten untuk tujuan penipuan (phishing), dan pada kasus Stored XSS, serangan dapat menyebar secara otomatis ke semua pengguna yang mengunjungi halaman tanpa interaksi apapun dari penyerang.

D. Implementasi Mitigasi

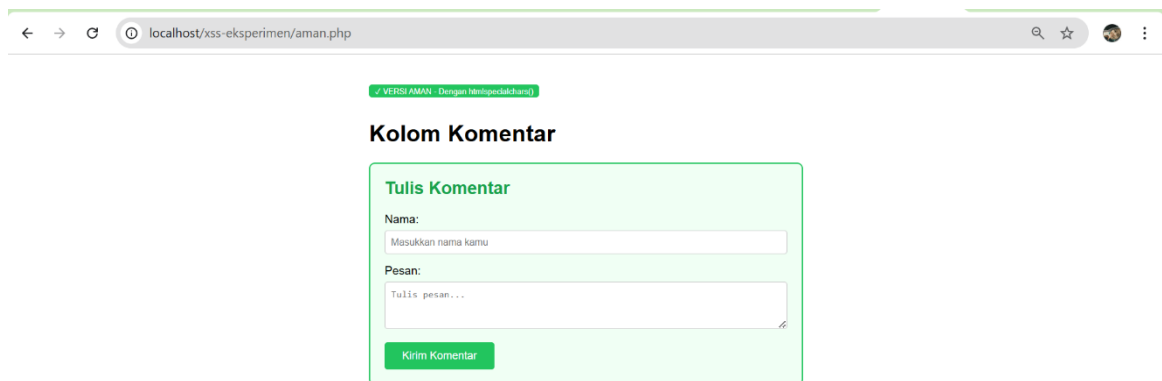
Solusi Utama: `htmlspecialchars()`

Fungsi `htmlspecialchars()` pada PHP adalah solusi paling efektif dan langsung untuk mencegah XSS. Fungsi ini mengonversi karakter-karakter khusus HTML menjadi HTML entities sehingga browser menampilkannya sebagai teks biasa, bukan mengeksekusinya sebagai kode:

```
// KODE PHP AMAN - DENGAN SANITASI htmlspecialchars()
$nama = htmlspecialchars($_POST['nama'], ENT_QUOTES, 'UTF-8');
$pesan = htmlspecialchars($_POST['pesan'], ENT_QUOTES, 'UTF-8');
```

```
// Ditampilkan ke HTML - AMAN karena sudah disanitasi
echo '<strong>' . $nama . '</strong>';
echo '<p>' . $pesan . '</p>';
```

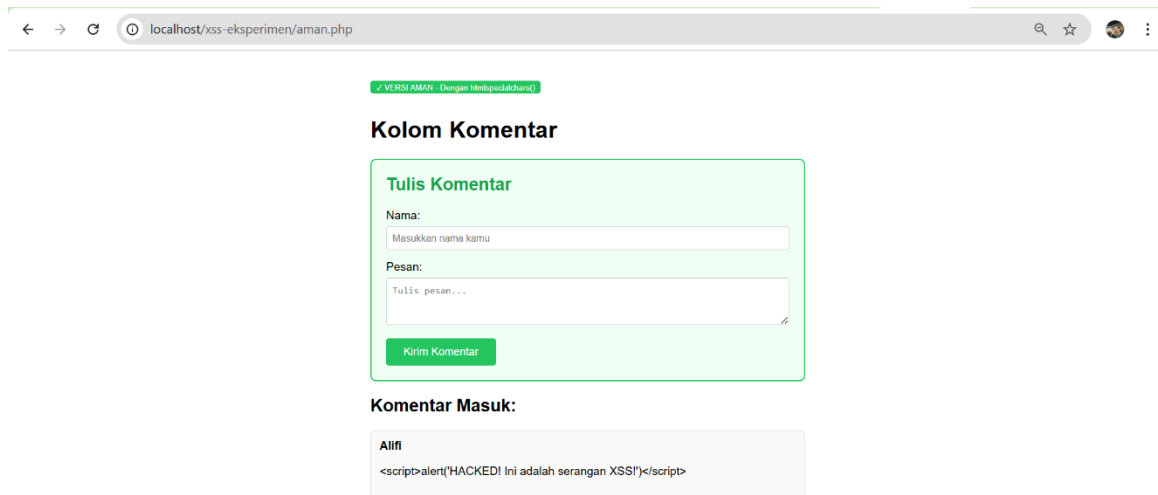
Mekanisme perlindungan bekerja melalui konversi karakter berbahaya. Ketika penyerang memasukkan `<script>alert('XSS')</script>`, fungsi `htmlspecialchars()` mengubahnya menjadi `<script>alert('XSS')</script>`. Browser kemudian menampilkan string tersebut sebagai teks literal, bukan mengeksekusinya sebagai kode JavaScript.



Gambar 6. Tampilan aman.php — halaman kolom komentar versi terproteksi



Gambar 7. Payload XSS yang sama dimasukkan ke versi aman — tidak ada pop-up, payload tampil sebagai teks biasa



Gambar 8. Bukti mitigasi berhasil: payload ditampilkan sebagai teks mentah, script tidak dieksekusi oleh browser

Rekomendasi Mitigasi Berlapis

Selain htmlspecialchars() sebagai lapisan utama, pendekatan pertahanan berlapis sangat direkomendasikan untuk meningkatkan postur keamanan aplikasi secara menyeluruh:

Lapisan	Mitigasi	Fungsi
Output Layer	htmlspecialchars()	Konversi karakter berbahaya jadi HTML entities — lapisan utama wajib
Input Layer	Validasi & whitelist input	Tolak input yang tidak sesuai format yang diharapkan
Header Layer	Content-Security-Policy (CSP)	Batasi sumber skrip yang diizinkan browser untuk dieksekusi
Cookie Layer	Flag HttpOnly pada cookie	Cegah JavaScript mengakses cookie sesi pengguna
Framework	Gunakan template engine	Framework modern (Laravel, React) secara otomatis melakukan escaping output

KESIMPULAN

Eksperimen yang dilakukan dalam artikel ini telah membuktikan secara empiris bahwa Cross-Site Scripting bukan sekadar ancaman teoritis. Melalui tiga tahap serangan menggunakan payload JavaScript pada kolom komentar yang tidak terproteksi, penulis berhasil menunjukkan bagaimana penyerang dapat memunculkan pop-up berbahaya, mencuri informasi cookie sesi pengguna, hingga merusak dan mengambil alih tampilan

seluruh halaman web semua hanya dengan memanfaatkan satu kelemahan fundamental: input pengguna yang ditampilkan tanpa sanitasi.

Fungsi `htmlspecialchars()` dengan parameter `ENT_QUOTES` dan encoding UTF-8, yang diimplementasikan pada versi aman (`aman.php`), terbukti mengeliminasi seluruh kerentanan tersebut secara efektif. Payload yang sama persis yang sebelumnya berhasil mengeksekusi kode JavaScript, kini hanya tampil sebagai teks mentah yang tidak berbahaya dibuktikan langsung melalui eksperimen perbandingan.

Pemahaman mendalam tentang cara kerja sebuah serangan merupakan prasyarat yang tidak dapat diabaikan untuk membangun pertahanan yang efektif. Pengembang yang memahami mengapa XSS dapat berhasil akan secara konsisten membuat keputusan implementasi yang benar: selalu sanitasi output, terapkan Content Security Policy, lindungi cookie dengan flag `HttpOnly`, dan manfaatkan mekanisme escaping bawaan dari framework modern. Eksperimen langsung dalam lingkungan yang terkontrol, sebagaimana yang dilakukan dalam artikel ini, adalah cara paling efektif untuk membangun intuisi keamanan yang nyata dan berkelanjutan.

REFERENSI

Gustiyo, A., Alwi, E. I., & Abdullah, S. M. (2024). Analisa kerentanan website terhadap serangan Cross-Site Scripting (XSS) metode penetration testing. *Cyber Security dan Forensik Digital*, 7(1), 25-33. <https://doi.org/10.14421/csecurity.2024.7.1.4432>

Gustiyo, A., et al. (2024). Penerapan teknik penetration testing terhadap Cross Site Scripting (XSS) dalam pengembangan website. *RABIT: Jurnal Teknologi dan Sistem Informasi Univrab*, 9(2), 262-270. <https://doi.org/10.36341/rabit.v9i2>

OWASP Foundation. (2021). OWASP Top 10:2021 - A03:2021 Injection. Open Web Application Security Project. https://owasp.org/Top10/A03_2021-Injection/

Sulistiyani, E., & Ananda, R. F. G. (2026). Analisis keamanan terhadap kombinasi XSS dan Social Engineering pada website. *Jurnal Informatika Polinema*, 12(2), 367-374. <https://doi.org/10.33795/jip.v12i2.8779>

The PHP Group. (2024). PHP Manual: `htmlspecialchars` - Convert special characters to HTML entities. PHP Documentation. <https://www.php.net/manual/en/function htmlspecialchars.php>

Wikipedia. (2024). Cross-site scripting. Wikimedia Foundation.
https://en.wikipedia.org/wiki/Cross-site_scripting

Alipiani Dwi Putri | NIM 312410691 | Kelas I241B | Universitas Pelita Bangsa